



WHITEPAPER

RougeChain: A Post-Quantum Layer 1 Blockchain

Version 2.2 -- 28 September 2026
rougechain.io · rougecoin.io

RougeChain is a post-quantum Layer 1 blockchain where every signature, every transaction, and every encrypted message is secured by NIST-approved lattice cryptography — not as a future upgrade, but as the foundation.

Abstract

RougeChain is a Layer 1 blockchain built on NIST-standardized post-quantum cryptographic primitives for signatures and encryption, with hash-based systems for proofs and commitments. Every transaction signature, block proposal, and encrypted message on the network uses ML-DSA-65 (FIPS 204) and ML-KEM-768 (FIPS 203), providing NIST Level 3 security -- equivalent to 192-bit classical strength -- against both classical and quantum adversaries. The chain incorporates zk-STARKs for shielded transfers and rollup batch proving, so signatures, encryption, and zero-knowledge proofs all rest on quantum-resistant assumptions. RougeChain combines a Proof-of-Stake consensus protocol with an application layer built into the node: an automated market maker, bridges to Base and Bitcoin, an NFT standard with protocol-level royalties, custom token issuance, WASM smart contracts, an end-to-end encrypted messenger and mail system, and a node-hosted social layer with signed posts and engagement.

Mainnet (`rougechain-mainnet-1`) is live. Since height 150 every block must carry a verified commit certificate -- precommits from at least two-thirds of stake -- for its parent, so finality is enforced by the protocol rather than reported by the producer. It is still an early-stage network: one validator produces blocks today and there is no fallback proposer, the bridges depend on operator-run relayers, and no external security audit has been completed. Section 11.10 lists these limitations in full.

Changes in v2.2

- **Contract randomness (3.1.3, 10.5, 11.10):** v2.1 said a player "cannot re-roll a transaction once sent". That was misleading: `host_random` can be ground by the sender, who signs many variants of a transaction offline and sends only a winning one. Height 170 (GAME_READY 3) adds `host_block_hash`, and games settle rolls from the hash of a block created after the player committed.

Changes in v2.1

Version 2.1 (28 September 2026) records the two protocol upgrades activated after v2.0:

- **Finality (3.1, 3.1.3, 11.5):** FINALITY_V2 is active from height 150 (Release 2a). Every block from 151 carries `parent_commit`, a verified precommit certificate for its parent, and `finalized_height` tracks verified certificates. Release 2b (fallback proposer, slashing, spreading stake across validators) is not done.
- **Contracts (3.1.3, 10.5):** GAME_READY (height 150) makes contract deployments and calls player-signed. GAME_READY 2 (height 160) lets contracts hold and move custom tokens and NFTs, create NFT collections and mint to players, and draw per-transaction randomness; cross-contract calls now apply their asset moves. Section 10.5 corrects v2.0's statement that sub-call balance changes were already applied.
- **State root (2.4, 3.2, 3.8):** from height 160 the header state root (v2) also covers NFT collections and ownership, contract code and contract storage.
- **DEX (5.4):** liquidity providers can withdraw accrued swap fees with "Collect fees", backed by a per-position fee ledger on each node.
- **Developer ecosystem (10):** SDK 1.9 (`rc.contracts`), browser extension 1.4 (contract signing), contract events over WebSocket, the contract query endpoint and the `loot_roll` example contract.
- **Limitations and roadmap (11.10, 12):** finality and both GAME_READY releases move to Shipped; the remaining limitations are restated.

Changes Since v1.8

Version 2.0 brings this paper in line with the network as deployed in September 2026:

- **Consensus (3.1, 3.1.3, 11.3-11.5):** documents the four mainnet protocol upgrades (heights 18, 49, 90 and 100), Release 1 proposer selection, the actual block cadence, and the real status of finality and slashing. The QRNG-weighted proposer selection and BFT finality described in v1.8 are not what mainnet runs.
- **Transactions (3.3):** replay protection by transaction identity and signed-payload binding replaces the per-account nonce description; the transaction-type count is corrected to 44.
- **Tokenomics (4):** balances are integer quanta ($1 \text{ XERGE} = 10^9 \text{ quanta}$); base-fee bounds and the bridge fee are stated.
- **Bridges (6, 11.6):** custody contracts and trust assumptions are stated plainly; the Bitcoin bridge is added.
- **Messenger (8):** real-time delivery events, deterministic direct-message IDs, recoverable delete, group management, avatars, and push notifications.
- **Social (9):** described as a node-hosted service rather than on-chain state.

- **Developer ecosystem (10):** current SDK, extension, MCP server, CLI and applications; mainnet routes that are not yet publicly open.
 - **Security (11.10):** new Current Limitations subsection.
 - **Roadmap (12):** split into Shipped, In Progress and Planned; BFT finality and STARK bridge verification are no longer marked complete.
-

Table of Contents

1. The Quantum Threat
 2. Post-Quantum Cryptography
 3. Protocol Architecture
 4. Tokenomics
 5. Decentralized Exchange
 6. Cross-Chain Bridges
 7. NFT Standard
 8. Encrypted Messenger and Mail
 9. Social Layer
 10. Developer Ecosystem
 11. Security Considerations
 12. Roadmap
-

1. The Quantum Threat

1.1 The Problem

Every major blockchain in production today -- Bitcoin, Ethereum, Solana, and others -- relies on elliptic curve cryptography (ECDSA or Ed25519) for transaction signing and address derivation. These schemes derive their security from the hardness of the elliptic curve discrete logarithm problem. In 1994, Peter Shor demonstrated a quantum algorithm that solves this problem in polynomial time, meaning a sufficiently large quantum computer would be able to forge any ECDSA or Ed25519 signature and spend any funds on any existing chain.

1.2 Timeline

Quantum computing is advancing rapidly. IBM, Google, and other organizations have demonstrated systems exceeding 1,000 qubits. While fault-tolerant quantum computers capable of breaking ECDSA remain years away, the cryptographic community has reached consensus

that migration must begin now for two reasons:

1. **Harvest now, decrypt later.** Adversaries can record blockchain transactions today and retroactively forge signatures once quantum hardware matures. Public keys exposed on-chain become permanent attack surfaces.
2. **Migration inertia.** Transitioning a live blockchain’s signature scheme is extraordinarily difficult. Chains that wait until quantum computers arrive will face emergency hard forks under adversarial conditions -- upgrading core cryptography while an attacker is actively exploiting the vulnerability they are trying to patch. Migration under attack is not an engineering challenge; it is a catastrophic failure scenario.

1.3 The Solution

RougeChain eliminates this risk class entirely. Rather than retrofitting post-quantum signatures onto a chain designed around elliptic curves, RougeChain is built from the ground up on NIST-standardized post-quantum primitives. There is no legacy cryptography to migrate away from.

2. Post-Quantum Cryptography

RougeChain uses two NIST-standardized algorithms, both operating at Security Level 3 (192-bit classical equivalent, 128-bit quantum equivalent).

2.1 ML-DSA-65 -- Digital Signatures (FIPS 204)

ML-DSA-65, derived from the CRYSTALS-Dilithium family, is used for all signature operations on RougeChain.

PARAMETER	VALUE
Standard	FIPS 204
Security Level	NIST Level 3
Public Key	1,952 bytes
Private Key	4,032 bytes
Signature	3,309 bytes
Underlying Problem	Module Lattice (Module-LWE)

Usage on RougeChain:

- Transaction signing (all transaction types)

- Block proposal signatures
- Validator vote attestations (prevote and precommit)
- Messenger message authentication

All signing is performed client-side. Private keys never leave the user's device.

2.2 ML-KEM-768 -- Key Encapsulation (FIPS 203)

ML-KEM-768, derived from the CRYSTALS-Kyber family, is used for key exchange in the encrypted messenger.

PARAMETER	VALUE
Standard	FIPS 203
Security Level	NIST Level 3
Public Key	1,184 bytes
Private Key	2,400 bytes
Ciphertext	1,088 bytes
Shared Secret	32 bytes
Underlying Problem	Module Lattice (Module-LWE)

Usage on RougeChain:

- E2E messenger key encapsulation
- Deriving AES-256-GCM session keys via HKDF

2.3 SHA-256 -- Hashing

SHA-256 is used for block hashes, transaction hashes, and Merkle computation. Grover's algorithm reduces SHA-256 to approximately 128-bit equivalent security against quantum adversaries, which remains well above the threshold for practical attacks.

2.4 zk-STARKs -- Zero-Knowledge Proofs

RougeChain incorporates a zk-STARK (Zero-Knowledge Scalable Transparent Argument of Knowledge) proof system for privacy-preserving transaction verification. Unlike zk-SNARKs, which rely on elliptic curve pairings vulnerable to Shor's algorithm, STARKs are built entirely on collision-resistant hash functions and are therefore **quantum-resistant by design**.

PROPERTY	ZK-SNARKS	ZK-STARKS (ROUGECHAIN)
Underlying math	Elliptic curve pairings	Hash functions + Reed-Solomon

PROPERTY	ZK-SNARKS	ZK-STARKS (ROUGECHAIN)
Trusted setup	Required	Not required (transparent)
Quantum resistance	No	Yes
Post-quantum safe	No	Yes

Implementation. RougeChain uses Meta's `winterfell` library, a production-grade Rust STARK prover and verifier. The system uses Blake3-256 as the commitment hash function, providing both quantum resistance and high performance. A WASM-compiled version of the prover (`core/wasm-prover/`) runs directly in the browser, enabling client-side proof generation for shielded operations without relying on a trusted server. Nodes verify proofs using the native Rust verifier.

Balance Transfer AIR. The first Algebraic Intermediate Representation (AIR) encodes a balance-transfer circuit that proves a token transfer is valid -- value is conserved, and the sender has sufficient funds -- without revealing the actual balances or transfer amount. The proof is generated over a finite field (128-bit prime), using a 3-column execution trace with the following transition constraints:

```
sender[next]    = sender[current] - per_step_amount
receiver[next]  = receiver[current] + per_step_amount
amount[next]    = amount[current]
```

The verifier only sees the final balances (public inputs). The initial balances and transfer amount remain private to the prover.

Security Parameters:

PARAMETER	VALUE
Field	128-bit prime (Goldilocks extension)
Hash function	Blake3-256
FRI queries	28
Blowup factor	8
Security level	~100 bits (conjectured)

Quantum Alignment. Because STARKs rely only on hash-based commitments (Merkle trees via Blake3), they share the same quantum-resistance profile as SHA-256: Grover's algorithm provides at most a quadratic speedup, reducing 256-bit hash security to ~128 bits -- still far beyond practical attack thresholds. This makes RougeChain's entire cryptographic stack quantum-resistant:

CRYPTOGRAPHIC PILLAR	ALGORITHM	QUANTUM-SAFE
Signatures	ML-DSA-65	Yes
Key exchange	ML-KEM-768	Yes
Hashing	SHA-256 / Blake3	Yes
Zero-knowledge proofs	zk-STARKs	Yes

ROLLUP BATCH AIR (PHASE 3)

The second AIR encodes a **batch transfer circuit** for zk-STARK rollups. It proves that N balance transfers executed off-chain are valid -- value is conserved for each transfer, no account is overdrafted, and the state root transitions correctly -- using a single proof that can be posted on-chain.

The rollup AIR uses a 5-column execution trace:

```
sender_before | sender_after | receiver_after | amount | running_hash
```

Transition Constraints:

1. **Value conservation:** $\text{sender_after} = \text{sender_before} - \text{amount}$
2. **Running hash accumulation:** $\text{running_hash}[i+1] = \text{running_hash}[i] + \text{sender_before}[i] * \text{amount}[i]$ (degree 2)
3. **Cross-check redundancy:** Additional constraint for proof soundness

Boundary Assertions:

COLUMN	STEP	VALUE
running_hash	First	pre_state_root
running_hash	Last	post_state_root

The rollup's state root is a SHA-256 Merkle tree computed from the sorted set of all account balances, using domain-separated hashing (`ROUGECHAIN_STATE_V1` for leaf nodes, `ROUGECHAIN_NODE_V1` for internal nodes).

This rollup root is separate from the **ledger state root** committed in every block header (Section 3.2). From height 18 the header root is a domain-separated SHA-256 over the sorted native XRGE, token and LP balances. From height 160 (GAME_READY 2) it is **state root v2**: `SHA-256("rougechain.stateroot.v2" || balance root || NFT collections || NFT ownership || contract code hashes || contract storage)`, where NFT collections contribute id, creator, minted count, maximum supply and frozen flag, and tokens contribute collection, id, owner and locked flag. A node that disagrees on any NFT owner or contract storage value therefore computes a different root and rejects the block at import, exactly as for a balance divergence. The root is a flat hash over sorted entries, not yet an incremental Merkle tree.

Rollup Accumulator. The daemon includes a `RollupAccumulator` that batches up to 32 transfers (or flushes after 5 seconds), executes them off-chain, computes the state root transition, generates a STARK proof, and self-verifies the proof before including it in the next L1 block. A single rollup proof of ~100 KB can attest to 32 transfer operations, reducing per-transaction amortized proof cost by 32×.

2.5 Comparison with Classical Schemes

PROPERTY	ECDSA (SECP256K1)	ED25519	ML - DSA - 65
Quantum-safe	No	No	Yes
NIST Standardized	No	No	Yes (FIPS 204)
Security Level	~128-bit classical	~128-bit classical	192-bit classical
Public Key Size	33 bytes	32 bytes	1,952 bytes
Signature Size	64 bytes	64 bytes	3,309 bytes
Signing Speed	~0.1 ms	~0.05 ms	~1 ms
Verification Speed	~0.3 ms	~0.1 ms	~0.5 ms

The trade-off is size: ML-DSA-65 signatures and keys are significantly larger than their classical counterparts. This is an inherent property of all lattice-based post-quantum schemes, not a flaw in RougeChain’s design. Section 3.7 details the concrete impact and mitigation strategy.

3. Protocol Architecture

3.1 Consensus: Proof of Stake

RougeChain uses a Proof-of-Stake consensus protocol. Any account that stakes at least **10,000 XRGE** becomes a validator; the staking key is the validator’s identity, and a node proposes blocks with the key stored in its `node-keys.json`.

Block Cadence. Mainnet genesis sets a **1,000 ms** block slot (`block_time_ms`; the daemon’s command-line default of 400 ms applies only to networks without that genesis setting). A block is produced only when the mempool holds at least one transaction -- the chain does not produce empty blocks. Block height therefore tracks network activity rather than wall-clock time (mainnet passed height 160 in late September 2026), and every protocol timer -- unbonding, jailing, missed-block thresholds -- is measured in blocks, not seconds.

Proposer Selection (Release 1). From height 100, each height has exactly one designated proposer:

```
eligible(H) = validators in the state after block H-1 with stake > 0 and jailed_until <= H
proposer(H) = the eligible validator with the largest stake
               (ties broken by the lowest raw public-key bytes)
```

A block signed by any other key is rejected at import before any state is modified, and a node refuses to seal a block for a height at which it is not the designated proposer. The producer records every proposal in a local proposal journal (`proposal-journal-db`) keyed by height and parent hash, so it never signs two different blocks for the same slot. `/api/stats` reports the activation height and the designated proposer for the next height.

Release 1 has **no fallback**: if the designated proposer is offline, no blocks are produced until it returns. Release 2 is split in two: Release 2a (verified finality) has been active since height 150, and Release 2b (skip certificates and a deterministic fallback proposer) is planned; see Section 12.

Earlier versions of this paper described a stake-weighted random selection seeded by the ANU Quantum Random Number Generator. That function exists in the codebase, but it was never enforced on mainnet -- before height 100 a block from any staked validator was accepted -- and it is not part of the Release 1 rule.

Finality (FINALITY_V2, active from height 150). Validators sign prevotes and precommits with their own ML-DSA-65 staking keys over a message that commits to a domain tag (`ROUGECHAIN_FINALITY_VOTE_V2`), the chain ID, the vote type, the height, the round and the block hash. A **finality proof** for a block is a set of such precommits whose stake reaches the quorum $\text{floor}(2 \times \text{total stake} / 3) + 1$ of that block's validator set. Nothing in a proof is taken on trust: a node verifies every signature and recomputes the voting stake, total stake and quorum from its own validator-set snapshot for that height. Each validator keeps a durable signing journal, so it can never sign two different blocks for the same slot, and votes and proofs travel between nodes over the peer layer.

This is **Release 2a** of the Release 2 design. From height 151 every block header carries `parent_commit`, the finality proof for its parent. A node refuses to import a block whose certificate is missing, is for a different parent, falls short of quorum or fails verification, and the producer does not seal a block until it holds a verified certificate for the previous one; importing block H therefore finalizes block H-1. `finalized_height` in the API now tracks the highest height with a verified certificate, rather than a value the producer sets for itself as it did before height 150 (the legacy vote path, which did not verify votes, applies only below height 150).

What finality does not yet add is liveness. Only one validator produces blocks, and the operator's validator holds a large majority of stake, so its own precommit reaches quorum on its own: a block is finalized as soon as it is produced, without waiting for other validators. **Release 2b** -- rounds, skip certificates, a deterministic fallback proposer, validator-admission hardening, and spreading stake across more independently run validators -- is designed but not implemented; until then an outage of the designated proposer still halts the chain.

Slashing. The protocol includes slashing: **10%** of a validator's stake per violation and a **20-block** jail, during which the validator is not eligible to propose. Before height 100, a validator that missed 50 block proposals was slashed automatically; this happened once on mainnet, at height 69. Release 1 **freezes missed-block accounting from height 100**, because with a single designated proposer every other validator would otherwise accumulate misses by design. Slashing based on evidence of misbehavior (for example, signing two blocks at one height) is planned as Release 3 and is not active.

Unbonding Period. When a validator unstakes, the amount enters an **unbonding queue** for **500 blocks**. Because blocks are produced only when there are transactions, this is a block count, not a fixed duration; its wall-clock length depends on network activity. During the unbonding period:

- Funds remain locked and cannot be transferred.
- The unbonding entry includes the delegator address, amount, and release height.
- At each new block, the node processes the unbonding queue and releases matured entries to the delegator's balance.
- This prevents stake-and-run attacks and ensures validators remain accountable for the blocks they participated in.

Fee Distribution. Transaction fees follow an EIP-1559-inspired dynamic fee model. The total fee consists of a **base fee** and a **priority fee**. **50%** of the base fee is burned; the remaining 50% flows into the distributable tip pool alongside the priority fee. The tip pool is split:

- **20%** to the block proposer.
- **70%** distributed to all active validators, weighted by their stake.
- **10%** allocated to the on-chain treasury, controlled by governance.

A **minimum tip floor** of 0.1 XRGE per block is enforced. If fees alone do not reach this threshold, the shortfall is drawn from the `__staking_rewards__` genesis reserve, so validators receive a baseline reward even when traffic is low.

The base fee starts at 0.1 XRGE, adjusts by at most $\pm 12.5\%$ per block toward a target of 10 transactions per block, and never falls below **0.001 XRGE**.

3.1.1 Governance

RougeChain includes an on-chain governance system for protocol parameters and treasury management.

Proposals. Any staked validator can create a governance proposal specifying a description, action type, quorum requirement, pass threshold, and timelock delay. Supported action types include `treasury_spend` (transfer funds from the on-chain treasury) and `parameter_change`.

Voting. Voting is balance-weighted with delegation support:

- Each voter's weight equals their own XRGE balance plus the balances of all accounts that have delegated to them.
- Delegators who vote directly are excluded from their delegate's weight (direct vote takes precedence).
- Proposals must meet both a quorum (minimum total votes) and pass threshold (percentage of yes votes) to succeed.

Delegation. Any account can delegate their voting power to another account via a `delegate` transaction. Delegation is one-to-one and can be revoked at any time via `undelegate`. Self-delegation is prohibited.

Timelock. Passed proposals enter a timelock period (measured in blocks) before they can be executed. This gives the community time to react to governance decisions.

Treasury. The on-chain treasury accumulates 10% of all priority fees. Treasury spending is controlled exclusively through governance proposals that pass quorum, threshold, and timelock requirements.

3.1.2 Multi-Signature Wallets

RougeChain supports M-of-N threshold multi-signature wallets at the protocol layer. Multi-sig wallets require M approvals from N designated signers before executing a transaction. All signatures use ML-DSA-65, making RougeChain multi-sig wallets quantum-resistant by default.

3.1.3 Protocol Upgrades

Protocol rules change through coordinated hard forks. Each upgrade is activated at a height compiled into the node software, and every validator must run the new release before that height. There is no on-chain upgrade governance yet. Mainnet has activated seven upgrades:

HEIGHT	UPGRADE	WHAT CHANGED
18	v2 ledger	Balances moved from floating-point values to integer quanta (1 XRGE = 10^9 quanta); block headers commit to a SHA-256 state root of the ledger; WASM contracts can hold and move XRGE.
49	F49	The ledger was migrated to its canonical state at height 48, with blocks 18-48 pinned by hash; bridge withdrawal records are created only when the burn actually succeeds; stake is registered only if its debit succeeds; block production commits atomically; matured unbonding is released before the state root is computed; peers sync only through verified block import, never by replacing the chain.
90	Transaction integrity	A transaction may be included only once (canonical-identity uniqueness), and V2 transactions must match the payload that was actually signed (Section 3.3).

HEIGHT	UPGRADE	WHAT CHANGED
100	Proposer selection, Release 1	One designated proposer per height (the largest eligible stake); missed-block accounting frozen; producer proposal journal (Section 3.1).
150	FINALITY_V2 (Release 2a) + GAME_READY	Verified finality: from block 151 each header carries <code>parent_commit</code> , a verified precommit certificate from at least two-thirds of stake for its parent, and blocks without one are refused (Section 3.1). Contract transactions become player-signed: <code>contract_deploy</code> and <code>contract_call</code> are valid only when signed by the deployer or caller, who pays the fee, and node-signed contract transactions are invalid. A deployed contract's address is the first 20 bytes of <code>SHA-256("rougechain/contract/v2" from 0 nonce 0 SHA-256(wasm))</code> . Deployment costs 10 XERGE; a call costs <code>gasLimit × 0.000001 XERGE</code> (Section 10.5).
160	GAME_READY 2	Contracts can hold and move custom tokens and NFTs, create their own NFT collections and mint to players, and draw per-transaction randomness (<code>host_random</code> , seeded by the parent block hash and the transaction hash). Addresses a contract supplies are canonicalised, and NFT owner checks compare canonical addresses. Cross-contract (multi-hop) calls have their XERGE, token and NFT moves applied. State root v2 commits to balances plus NFT collections and ownership, contract code hashes and contract storage (Section 2.4).
170	GAME_READY 3	<code>host_block_hash(height)</code> : contracts can read the hash of a finished block up to 256 back, so games commit in one call and settle from a block that did not exist when the player committed. Fixes the sender-grinding weakness of one-step <code>host_random</code> rolls.

3.2 Block Structure

Each block consists of a versioned header, a list of transactions, the proposer's ML-DSA-65 signature, and a SHA-256 hash.

BlockHeaderV1:

FIELD	TYPE	DESCRIPTION
version	u32	Protocol version (currently 1)
chain_id	String	Network identifier
height	u64	Block number
time	u64	Timestamp in milliseconds
prev_hash	String	SHA-256 hash of the previous block
tx_hash	String	SHA-256 hash of all transactions
proposer_pub_key	String	ML-DSA-65 public key of the proposer

FIELD	TYPE	DESCRIPTION
state_root	Option<String>	SHA-256 commitment to the balance ledger (present from height 18); from height 160, state root v2 also covers NFTs and contract state
parent_commit	Option<FinalityProof>	Verified precommit certificate for the parent block (required from height 151; absent before)

BlockV1:

FIELD	TYPE	DESCRIPTION
version	u32	Protocol version
header	BlockHeaderV1	Block metadata
txs	Vec<TxV1>	Ordered list of transactions
proposer_sig	String	ML-DSA-65 signature over header
hash	String	SHA-256(header_bytes proposer_sig)

3.3 Transaction Model

RougeChain defines 44 transaction types within a unified transaction structure:

TxV1:

FIELD	TYPE	DESCRIPTION
version	u32	Transaction version
tx_type	String	One of the 44 transaction types
from_pub_key	String	Sender's ML-DSA-65 public key
nonce	u64	Sender counter (replay protection comes from transaction identity; see below)
payload	TxPayload	Type-specific data
fee	f64	Fee in decimal XRG as signed; converted to integer quanta when applied (Section 4.1)
sig	String	ML-DSA-65 signature
signed_payload	Option<String>	V2 canonical signed payload

Supported Transaction Types:

CATEGORY	TYPES
Transfers	<code>transfer</code>
Tokens	<code>create_token</code>
Staking	<code>stake</code> , <code>unstake</code> , <code>slash</code>
AMM / DEX	<code>create_pool</code> , <code>add_liquidity</code> , <code>remove_liquidity</code> , <code>swap</code>
Bridge	<code>bridge_mint</code> , <code>bridge_withdraw</code>
NFTs	<code>nft_create_collection</code> , <code>nft_mint</code> , <code>nft_batch_mint</code> , <code>nft_transfer</code> , <code>nft_burn</code> , <code>nft_lock</code> , <code>nft_freeze_collection</code>
Shielded	<code>shield</code> , <code>shielded_transfer</code> , <code>unshield</code>
Governance	<code>create_proposal</code> , <code>cast_vote</code> , <code>execute_proposal</code> , <code>delegate</code> , <code>undelegate</code>
Multi-Sig	<code>multisig_create</code> , <code>multisig_approve</code> , <code>multisig_submit</code>
Allowances	<code>approve</code> , <code>transfer_from</code> , <code>token_approve</code> , <code>token_transfer_from</code>
Tokens (extended)	<code>mint_tokens</code> , <code>token_airdrop</code> , <code>token_lock</code> , <code>token_unlock</code>
Token Staking	<code>create_staking_pool</code> , <code>token_stake</code> , <code>token_unstake</code>
Contracts	<code>contract_deploy</code> , <code>contract_call</code>
Orders	<code>place_limit_order</code> , <code>cancel_limit_order</code>

Note: The faucet is implemented as a `transfer` transaction with a `faucet: true` flag in the payload, signed by the node operator's key. It is not a separate transaction type, and it is disabled on mainnet.

Client-Side Signing. All transactions are signed on the user's device using their ML-DSA-65 private key. The signed transaction is then submitted to any node via the REST API. At no point does a private key leave the client. The v2 transaction endpoints enforce this by design -- they accept only pre-signed payloads and reject empty or invalid signatures.

V2 Signed Payload Format. V2 transactions carry `signed_payload`: the sorted-key JSON that the user actually signed. Since height 90 the node derives the transaction's security-relevant fields (recipient, amount, fee, token, and so on) from that signed payload and rejects any transaction whose top-level fields differ from it, both at API ingress and at block import. Two signed formats are bound this way: the flat format used by the REST API and SDK, and the envelope format used by the CLI. Before height 90 only the signature over `signed_payload` was checked.

Transaction Uniqueness. Since height 90 every transaction has a canonical identity hash, and a transaction whose identity is already included in an earlier block is rejected at mempool admission, when a block is produced, and at block import. The index of included identities (`tx-`

seen-db) is rebuilt from the chain at startup. This replaced an earlier per-account nonce check that was not enforced at block import; the nonce field is not required to be sequential.

3.4 Storage

RougeChain uses the Sled embedded database for persistent state, providing:

- **$O(1)$ block lookups** by height
- **$O(1)$ transaction lookups** by hash via a dedicated tx_index tree
- **Range queries** for block scanning and pagination
- **Persistent state** for shielded supply, faucet cooldowns, and chain metadata
- **Atomic writes** for consistency during block import

The following stores are maintained independently:

STORE	DATABASE	CONTENTS
Chain	chain-db	Blocks indexed by height
Tx Index	chain-db (tx_index tree)	$O(1)$ transaction hash → block height lookup
State	chain-db (state tree)	Persistent chain state (shielded supply, faucet cooldowns)
Nonce	nonce-db	Per-account nonce tracking
Included Transactions	tx-seen-db	Canonical identities of included transactions (replay protection, from height 90)
Proposal Journal	proposal-journal-db	The producer's signed proposals by height and parent (from height 100)
Validators	validators-db	Stake, slash count, jail status, entropy contributions
Pools	pools-db	AMM liquidity pool state
NFT Collections	nft-collections-db	Collection metadata and configuration
NFT Tokens	nft-tokens-db	Individual token ownership and attributes
Token Metadata	token-metadata-db	Custom token metadata (name, image, socials)
Finality Records	finality-db	Verified FINALITY_V2 proofs by block height (from height 150; earlier records are legacy and informational)
Event Index	indexer-db	Multi-index event store (by address, type, token, block)
Transaction Receipts	receipts-db	Post-inclusion status, logs, and gas used
Contracts	contracts-db	WASM bytecode, metadata, and contract storage

STORE	DATABASE	CONTENTS
Messenger	messenger-db	Wallets, conversations, messages with participant and conversation-message indexes
Mail	mail-db	Mail messages and per-recipient folder labels
Name Registry	name-registry-db	Bidirectional name↔wallet mappings with atomic registration (compare-and-swap)
Social	social-db	Posts, likes, reposts, follows, comments, play counts, timeline indexes

Additional off-chain stores (JSON-backed) handle bridge claims and withdrawal requests.

3.4.1 Event Indexer

RougeChain includes a built-in event indexer that maintains secondary indexes over all on-chain transactions. The indexer supports paginated queries by:

- **Address** — all transactions involving a given public key (as sender or recipient)
- **Transaction type** — filter by protocol transaction type
- **Token symbol** — activity feed for a specific token
- **Block height** — all events within a given block

The indexer automatically backfills from the chain store on startup and indexes new blocks in real-time during block production and peer import. API endpoints are exposed at `/api/indexer/{address|type|token|block}/:param`.

3.4.2 Observability

The node exposes a `/metrics` endpoint in Prometheus text exposition format, providing real-time gauges for:

- Block height, finalized height, and base fee
- Validator count, total staked, and mempool size
- Peer count, WebSocket client count, and indexed event count
- Lifetime fees collected and burned

A `docker-compose.yml` with an optional `monitoring` profile provisions a Prometheus instance pre-configured to scrape the node.

3.5 Networking

Nodes communicate over HTTP with the following mechanisms:

Peer Discovery. Nodes register with known peers via `POST /api/peers/register` and periodically fetch the peer list via `GET /api/peers`. New peers discovered transitively are added automatically.

Block Synchronization. Nodes poll peers for new blocks with adaptive frequency (10-60 seconds). On initial sync, blocks are fetched in batches of 1,000. Failed syncs trigger exponential backoff up to 10 minutes.

Block Propagation. When a node produces or imports a new block, it broadcasts asynchronously to all known peers via `POST /api/blocks/import` with a 5-second timeout per peer.

Transaction Broadcast. Submitted transactions are propagated to peers via `POST /api/tx/broadcast`.

Current Mainnet Topology. Mainnet runs on two nodes: the producing validator and a second node that follows it without producing blocks. On the public mainnet endpoint (`api.rougechain.io`), the unauthenticated peer-write routes `POST /api/blocks/import`, `POST /api/peers/register` and `POST /api/tx/broadcast` are closed at the reverse proxy; other nodes sync by pulling blocks. Node-to-node communication is HTTP polling; a dedicated peer-to-peer transport is not yet implemented.

Rate Limiting. The API implements three-tier rate limiting applied to all endpoints:

- **Tier 1 (Validators):** Elevated throughput for staked validators. Authentication requires three headers: `X-Validator-Key` (public key), `X-Validator-Sig` (ML-DSA-65 signature over the timestamp), and `X-Validator-Ts` (Unix millisecond timestamp). The signature is verified via `pqc_verify`, and the timestamp must be within a 30-second drift window. This prevents spoofing of validator status.
- **Tier 2 (Peers):** Moderate limits for registered peer nodes, identified by socket address.
- **Tier 3 (General):** Configurable rate limits for public API consumers, with separate limits for read (GET) and write (POST) operations.

3.6 Mempool

The mempool holds up to **2,000 pending transactions** in a hash map keyed by transaction hash. When the mempool is full, the **lowest-fee transaction is evicted** — new transactions must have a higher fee than the current minimum to be admitted. This fee-priority eviction prevents an attacker from flooding the mempool with low-fee spam to displace legitimate transactions. Duplicate transactions are rejected. P2P broadcast transactions undergo full ML-DSA-65 signature verification before admission.

All transactions in the mempool are verified (including parallel signature checks via Rayon) and included in the next block produced by the local node, then drained.

3.7 Signature Size and Scaling

Post-quantum signatures are larger than classical signatures. This is an inherent property of lattice-based cryptography and applies to every NIST-standardized PQ signature scheme. RougeChain confronts this tradeoff directly.

Size Reality:

SCHEME	PUBLIC KEY	SIGNATURE	PER-TX OVERHEAD
ECDSA (Ethereum)	33 bytes	64 bytes	~97 bytes
Ed25519 (Solana)	32 bytes	64 bytes	~96 bytes
ML-DSA-65 (RougeChain)	1,952 bytes	3,309 bytes	~5,261 bytes

A RougeChain transaction carries approximately **54× more signature data** than an Ethereum transaction. This directly impacts block size, storage growth, and network bandwidth.

Impact Analysis:

METRIC	CLASSICAL CHAIN	ROUGECHAIN	FACTOR
Per-transaction overhead	~100 bytes	~5.3 KB	~54×
1,000-tx block (signatures only)	~100 KB	~5.3 MB	~54×
Annual storage (1 tx/sec)	~3 GB	~162 GB	~54×

This is significant but manageable. Modern consumer hardware handles these volumes, and the numbers are comparable to chains with high-throughput blocks (Solana averages 100+ MB/min).

Mitigation Strategy:

1. **Transaction batching.** Multiple operations can share a single signature by bundling them into a single transaction payload, amortizing the 3.3 KB cost across many operations.
2. **Network-layer compression.** Block data is compressible at the transport layer. Signature bytes exhibit low entropy and compress well with standard algorithms (zstd/lz4), reducing actual bandwidth by 30-50%.
3. **Pruning and archival.** Full signature data is required only for verification. Fully validated blocks can be pruned to headers + state roots for archival nodes.
4. **zk-STARK rollups (Phase 3).** The primary long-term scaling path. Off-chain transactions are batched and verified via a single STARK proof posted on-chain. A rollup proof of ~100 KB can attest to thousands of transactions, making per-transaction amortized cost smaller than even classical chains.

Context. This size increase is the cost of quantum resistance. Every post-quantum signature scheme — including SLH-DSA (SPHINCS+), which produces even larger signatures at ~17 KB — faces this tradeoff. Chains that defer PQ migration will eventually pay the same cost, but under

adversarial conditions during an emergency hard fork. RougeChain pays it now, by design.

3.8 Design Philosophy: Vertically Integrated L1

RougeChain intentionally adopts a vertically integrated L1 design, building a DEX, NFT standard, bridges, messenger, name service, mail system and social layer into the node itself. A new chain with no dApps has no users, and no users means no dApps -- by shipping core primitives with the node, RougeChain offers usable applications without waiting for third parties to build them.

These components are not all on-chain. The DEX, NFTs, tokens, staking, governance, shielded transfers, contracts and bridge mints and burns are transactions in blocks. Token, LP and XERGE balances are covered by the state root from height 18; NFT collections and ownership and contract code and storage are covered from height 160 (state root v2, Section 2.4). The messenger, mail, name registry and social layer are **node-hosted services**: every write is authenticated with an ML-DSA-65 signature, but the data is stored by the node in its own databases and is not part of consensus state.

This increases protocol complexity but enables immediate usability without reliance on external smart contract layers or third-party infrastructure. Future iterations may modularize components as the ecosystem matures and developer tooling enables permissionless application deployment.

3.9 Shielded Transactions

RougeChain supports private value transfers through a note-based shielded transaction system built on its zk-STARK proof infrastructure. Shielded transactions hide transfer amounts and account balances behind cryptographic commitments while preserving on-chain verifiability.

Commitment Scheme. A shielded note is represented as a SHA-256 commitment:

```
commitment = SHA-256("ROUGECHAIN_COMMITMENT_V1" || value || owner_pubkey || randomness)
```

The `value` and `randomness` are private — only the commitment hash is stored on-chain. Two notes with the same value produce different commitments due to the random blinding factor.

Double-Spend Prevention. Each note has a unique nullifier derived from its randomness:

```
nullifier = SHA-256("ROUGECHAIN_NULLIFIER_V1" || randomness || commitment)
```

When a note is consumed, its nullifier is published on-chain. If a nullifier already exists in the nullifier set, the transaction is rejected as a double-spend. The nullifier does not reveal which commitment it corresponds to.

STARK Proof of Value Conservation. Every shielded transfer includes a zk-STARK proof that the sum of input note values equals the sum of output note values plus the fee:

$$\text{input_value} = \text{output_value_1} + \text{output_value_2} + \text{fee}$$

The proof uses a 5-column, 64-row arithmetic execution trace:

COLUMN	PURPOSE
input_value	Value of consumed note (constant)
output_1	Recipient value (constant)
output_2	Change value (constant)
fee	Transaction fee (public)
bit_accumulator	Bit-decomposition range check

The 64-row trace performs MSB-first bit decomposition of the input value, proving it fits within 64 bits. This prevents overflow attacks where an adversary could create notes with astronomical values by exploiting finite field arithmetic.

Transaction Lifecycle:

1. **Shield** (`shield`): Deducts XRGE from the sender's public balance and stores a commitment on-chain. The note details (value, randomness) are kept by the sender.
2. **Shielded Transfer** (`shielded_transfer`): Consumes one or more input notes by publishing their nullifiers, creates new output commitments, and includes a STARK proof that value is conserved. Neither the input nor output values are revealed.
3. **Unshield** (`unshield`): Publishes a nullifier to consume a shielded note and credits the equivalent XRGE to the sender's public balance, with a STARK proof of note ownership.

Supply Tracking. The total shielded supply is tracked persistently in the `state` sled tree and updated atomically during shield/unshield operations. The API exposes a supply breakdown:

METRIC	SOURCE
Circulating Supply	Total supply – burned – shielded
Shielded Pool	Cumulative shield minus unshield amounts
Burned Supply	Balance of the burn address

Proof Size Tradeoff. Shielded transfer STARK proofs are approximately 50–100 KB each, compared to ~5.3 KB for a standard transfer (including the 3.3 KB ML-DSA signature). This is an inherent characteristic of STARKs — transparency and quantum resistance come at the cost of larger proofs. Future rollup batching (Phase 3) will amortize this by aggregating hundreds of shielded proofs into a single on-chain proof.

4. Tokenomics

4.1 XRGE (RougeCoin)

XRGE is the native token of RougeChain. Every fee paid, every stake deposited, and every trade executed on RougeChain is denominated in XRGE, and every XRGE transfer is authorized with an ML-DSA-65 signature.

XRGE serves four functions on the network:

- **Transaction fees.** Every operation on RougeChain requires XRGE, creating baseline demand proportional to network activity.
- **Validator staking.** XRGE is staked to participate in consensus. Validators earn fee revenue proportional to their stake, aligning economic incentive with network security.
- **Base trading pair.** XRGE is the default routing token in the DEX's multi-hop swap engine, making it the liquidity backbone of on-chain trading.
- **Governance weight.** Staked XRGE determines voting power in protocol decisions, ensuring that those most invested in the network's security have the strongest voice.

PROPERTY	VALUE
Name	RougeCoin
Symbol	XRGE
Maximum Supply	36,000,000,000
Smallest Unit	1 quantum = 10^{-9} XRGE

Integer Quanta. Since the v2 ledger upgrade at height 18 (Section 3.1.3), balances are stored as integers in **quanta**, where 1 XRGE = 1,000,000,000 quanta. Amounts and fees are written in decimal XRGE in signed transactions and are converted to quanta when a transaction is applied, so ledger arithmetic is exact.

4.2 Fee Schedule

All operations on RougeChain require an XRGE fee. The fee schedule is designed to be accessible for common operations while discouraging spam on higher-impact actions.

OPERATION	FEE (XRGE)
Transfer	0.1
Token creation	100
Liquidity pool creation	10
Token swap	0.1

OPERATION	FEE (XRGE)
NFT collection creation	50
NFT mint	5
NFT batch mint	5 per token
NFT transfer	1
NFT lock / unlock	0.1
NFT collection freeze	0.1
Shield (public → private)	1
Shielded transfer	1
Unshield (private → public)	1
Bridge withdrawal (any asset)	0.1 (exactly)

Note: The v2 client-signed API endpoints enforce a minimum fee of **1.0 XRGE** for transfers, swaps, and liquidity operations to discourage spam. The base protocol fees listed above apply at the consensus layer; the API layer may enforce higher minimums.

4.3 Fee Distribution

RougeChain uses an **EIP-1559-inspired dynamic fee model**. Each transaction fee consists of a **base fee** and a **priority fee** (tip):

- **50%** of the base fee is **burned**, permanently reducing the circulating supply. This makes XRGE deflationary under sustained network activity. The remaining 50% of the base fee flows into the tip pool alongside priority fees.
- The **tip pool** is distributed as follows:
 - **20%** to the block proposer as a direct reward for block production.
 - **70%** to all active validators, distributed proportionally to their staked XRGE.
 - **10%** to the on-chain treasury, governed by protocol governance.
- A **minimum tip floor** of 0.1 XRGE per block is enforced. When collected fees fall below this threshold, the shortfall is subsidised from the `__staking_rewards__` genesis allocation, guaranteeing validators a baseline return even during low-traffic periods.

The base fee adjusts dynamically based on block utilization. If a block holds more than the target of 10 transactions, the base fee increases by up to 12.5%; if it holds fewer, it decreases by up to 12.5%. The base fee starts at 0.1 XRGE and never falls below a floor of 0.001 XRGE, which is where it stood on mainnet in September 2026.

This model incentivizes block production, passive staking, and treasury accumulation for ecosystem funding.

4.4 Deflationary Mechanism

RougeChain defines a permanent burn address:

```
XRGE_BURN_0x0000000000000000000000000000000000000000000000000000000000000000DEAD
```

Tokens sent to this address are irreversibly destroyed and tracked on-chain. Any user or smart contract may burn tokens, permanently reducing the circulating supply.

5. Decentralized Exchange

RougeChain includes a native automated market maker (AMM) built directly into the protocol layer, requiring no smart contracts.

5.1 Constant Product Formula

The AMM uses the constant product invariant popularized by Uniswap V2:

$$x * y = k$$

Where x and y are the reserves of the two tokens in a pool, and k is a constant that increases only when liquidity is added.

5.2 Swap Mechanics

Each swap incurs a **0.3% fee** applied to the input amount:

$$\text{amount_out} = (\text{amount_in} * 997 * \text{reserve_out}) / (\text{reserve_in} * 1000 + \text{amount_in} * 997)$$

The fee remains in the pool, increasing k and benefiting liquidity providers.

Slippage protection is enforced via a `min_amount_out` parameter. If the calculated output falls below this threshold, the swap is rejected.

Price impact is calculated as the percentage difference between the spot price and the effective execution price, giving users visibility into large-order effects.

5.3 Multi-Hop Routing

When a direct pool does not exist between two tokens, the AMM performs breadth-first search (BFS) across all pools to find a valid route. Multi-hop swaps execute atomically -- either all legs succeed or the entire swap reverts.

5.4 Liquidity Provision

Liquidity providers deposit paired tokens into a pool and receive LP tokens representing their share.

- **Minimum liquidity:** The first 1,000 LP tokens are permanently locked to prevent share manipulation.
- **Adding liquidity:** Amounts must be proportional to existing reserves. LP tokens minted: $\min(\text{amount_a} / \text{reserve_a}, \text{amount_b} / \text{reserve_b}) * \text{total_lp_supply}$.
- **Removing liquidity:** LP tokens are burned and the proportional share of both reserves is returned.
- **Collecting fees:** the 0.3% swap fee accrues to liquidity providers inside the pool, and a provider can withdraw what it has earned with **Collect fees** without touching its deposit. Collecting is an ordinary `remove_liquidity` of exactly the LP tokens that represent the position's fee growth; there is no separate fee transaction and no consensus change. To size it, each node keeps an off-consensus fee ledger per LP position: the deposit is recorded as a basis in share-value units, where one LP token is worth $\sqrt{(\text{reserveA} * \text{reserveB})} / \text{LP supply}$ (a value that swap fees raise and adding or removing liquidity does not), and withdrawals are counted against earnings first, so collecting fees leaves the deposit's basis intact. A position's earnings are exposed at `GET /api/pool/:id/earnings/:owner`. The ledger is maintained from pool events (and rebuilt from them once on nodes that predate it) and is not part of the state root; it only informs the client how much to withdraw.

There is no protocol fee: the whole 0.3% stays with liquidity providers.

5.5 Pool Creation

Any user may create a pool by specifying two tokens and initial deposit amounts. The pool ID is derived from the sorted token pair (e.g., `TOKENA-TOKENB`). A 10 XRGE fee is charged.

6. Cross-Chain Bridges

RougeChain operates two bridges: one to **Base** (Ethereum L2) for ETH, USDC and XRGE, and one to **Bitcoin** for BTC. Deposits are verified against the source chain before wrapped assets are minted on RougeChain; withdrawals burn the wrapped asset on RougeChain and are paid out on

the destination chain by an operator-run relayer. Both bridges rely on operator trust, which Section 6.7 sets out.

6.1 Multi-Network Support

The bridge dynamically selects the correct chain configuration based on the daemon's `/bridge/config` response:

NETWORK	CHAIN ID	XRGE CONTRACT	USDC CONTRACT
Base Mainnet	8453	0x147120...24317	0x833589...02913
Base Sepolia	84532	0xF9e744...Efa27E	0x036CbD...3dCF7e

6.2 Bridge In (Base → RougeChain)

1. The user sends ETH, USDC, or XRGE to the bridge contracts on Base.
2. The user submits the Base transaction hash to RougeChain's bridge API, or the node's deposit watcher detects the deposit and claims it automatically.
3. The RougeChain node performs **two-layer verification**:
 - **Layer 1 (Receipt Verification)**: Queries the Base RPC for the transaction receipt, verifying tx status (`0x1`), recipient matches the bridge contract, sender matches the claimed address, and the minimum confirmation depth is met (`QV_BRIDGE_MIN_CONFIRMATIONS` , default `6`).
 - **Layer 2 (Commitment Nullifier)**: Computes a SHA-256 commitment hash of the deposit (`SHA-256("ROUGECHAIN_BRIDGE_V1" || tx_hash || evm_address || amount)`) and checks it against the nullifier set. Duplicate commitments are rejected.
4. For contract deposits, the node additionally verifies the deposit event in the transaction logs, matching the indexed sender address.
5. Upon verification, the node creates a `bridge_mint` transaction, minting the wrapped asset to the user's RougeChain address. `bridge_mint` is authorized by the node operator's key (Section 6.7).

Replay protection: Each deposit commitment hash is stored persistently. A deposit can only be claimed once, even across node restarts.

EVM Signature Verification. Manual bridge claims require an ECDSA signature from the wallet that sent the deposit transaction. The claim message follows a canonical format (`RougeChain bridge claim\nTx: {hash}\nRecipient: {pubkey}`), and the signature is verified via `ecrecover` to ensure the claimant is the original depositor.

6.3 Multi-Asset Bridge

ASSET	BASE CHAIN	ROUGECHAIN WRAPPED	CUSTODY CONTRACT
ETH	Native ETH	qETH	RougeBridge (<code>0xc09C7...35d83</code>)

ASSET	BASE CHAIN	ROUGECHAIN WRAPPED	CUSTODY CONTRACT
USDC	ERC-20	qUSDC	RougeBridge (0x0c09C7...35d83)
XRGE	ERC-20 (Base)	XRGE (L1)	BridgeVaultV2 (0x7BB107...35Cd2D)

For ERC-20 tokens, the bridge parses `Transfer` event logs from the transaction receipt to determine the deposited amount, rather than relying on the transaction's ETH value field. qETH and qUSDC use 6 decimals on RougeChain; USDC is bridged 1:1, and ETH amounts are scaled by 10^{12} from its 18 decimals.

6.4 Bridge Out (RougeChain → Base)

1. The user signs a `bridge_withdraw` request with ML-DSA-65. The signature covers the operation type, amount, fee, token symbol, destination EVM address, sender, timestamp and nonce, and the node executes only these signed values; unsigned request fields must match them or the request is rejected. The fee must be exactly **0.1 XRGE** for every asset.
2. The node burns the wrapped asset. A withdrawal record is created only if the burn succeeded in an accepted block; a withdrawal whose burn fails produces no payout record.
3. The relayer pays out on Base: XRGE through BridgeVaultV2 `release()`, qETH through RougeBridge `releaseETH`, and qUSDC through RougeBridge `releaseERC20`. Each payout carries a canonical ID (`keccak256` of the RougeChain withdrawal ID), and the contracts' `processedL1Tx` record prevents a second payout for the same withdrawal.
4. The node marks the withdrawal fulfilled only after verifying the matching release event on Base at the required confirmation depth.

RougeBridge can hold large withdrawals in an on-chain timelock; a queued withdrawal is fulfilled when the timelock executes. Automatic refunds are disabled: a withdrawal is never refunded while its payout status is paid, queued or ambiguous.

6.5 Hardened Relayer

The bridge relayer implements the following safeguards for withdrawal processing:

- **Payout-state check first:** Before any release, the relayer checks the contract's `processedL1Tx` record and fails closed if the state cannot be read; an already-processed withdrawal is reconciled, never paid again.
- **Asset routing:** Each wrapped asset has an explicit payout route; unsupported assets are not paid.
- **Nonce management:** Manual nonce tracking prevents stuck transactions from blocking the queue.
- **Retry with exponential backoff:** Failed RPC calls are retried up to 3 times (1s → 2s → 4s delays).
- **Gas estimation:** Uses `estimateGas` with a 10% buffer instead of hardcoded gas limits.

- **Preflight checks:** On start, the relayer checks chain ID, contract code, ownership, pause state and token configuration.
- **Graceful shutdown:** SIGTERM/SIGINT handlers wait for in-flight transactions before exiting.
- **Confirmation depth:** Waits for the greater of its own setting and the node's minimum (default 6 confirmations) before asking the node to mark a withdrawal fulfilled.

6.6 Security Model

The Base bridge uses two custody contracts with different trust properties:

- **BridgeVaultV2 (XRGE).** Roles are split. The owner is a Safe multisig, which alone can change the relayer, adjust caps, pause the vault, or start an emergency withdrawal (subject to a 48-hour timelock). The relayer is a hot key that can only call `release()`, bounded by an on-chain per-transaction cap and a rolling 24-hour limit, and is blocked while the vault is paused. A compromised relayer key can therefore release at most the remaining 24-hour limit before the owner rotates or pauses it.
- **RougeBridge (ETH and USDC).** The owner is a single hot key held by the operator, and the same key signs releases. A 2-of-3 Safe acts as guardian and can cancel timelocked withdrawals, but below the timelock threshold the owner key can move the contract's funds. Moving RougeBridge ownership to a multisig is pending.

Deposit and withdrawal verification layers:

- **EVM receipt verification** against Base RPC (tx status, recipient, sender, confirmations).
- **SHA-256 commitment nullifiers** prevent double-claiming deposits.
- **Deposit event verification** for contract deposits (log parsing + sender matching).
- **ECDSA signature verification** (ecrecover) to authenticate depositors on manual claims.
- **Chain ID validation** to prevent cross-chain replay.
- **Signed withdrawal intent** (ML-DSA-65) bound to the executed amount, fee, asset and destination.
- **Burn-gated payout records** and **on-chain payout verification** before fulfillment.
- **Relayer authentication** via shared secret for withdrawal fulfillment.

6.7 Trust Assumptions

The bridges' trust model is explicit. The following table documents what is verified cryptographically versus what requires operational trust:

COMPONENT	VERIFICATION METHOD	TRUST ASSUMPTION
Deposit occurred on Base	EVM receipt verification via RPC	Relies on Base RPC endpoint returning honest data
Depositor identity	ECDSA ecrecover of signed claim	Cryptographic — no trust required
Double-claim prevention	SHA-256 commitment nullifier set	Cryptographic — no trust required

COMPONENT	VERIFICATION METHOD	TRUST ASSUMPTION
Contract deposits	Deposit event log parsing	Relies on correct contract deployment
Minting on RougeChain	<code>bridge_mint</code> signed by the node operator's key	Trusted operator: this key can mint wrapped assets
XRGE custody	BridgeVaultV2: Safe owner, capped hot relayer	Relayer compromise bounded by on-chain caps
ETH/USDC custody	RougeBridge: single hot EOA owner	Trusted operator: owner key compromise could drain the contract
Withdrawal fulfillment	Relayer executes on Base; node verifies the release event	Requires the operator-run relayer to be online and honest
Cross-chain consistency	Chain ID validation	Cryptographic — no trust required

Current model: The Base bridge is an **operator-trusted custody bridge**. Deposits are verified against Base before minting, but minting authority, the relayers and the RougeBridge owner key are all held by the operator.

Target model: The next step is BridgeVaultV3 (Section 12), in which releases on Base require M-of-N ML-DSA-65 authorization of withdrawal data derived from RougeChain, removing the single hot key from the XRGE release path. It depends on FINALITY_V2, which has been active on mainnet since height 150 (Section 3.1); V3 itself has not been deployed and is waiting on an external audit. Fully trustless operation through STARK proofs of Base block headers remains a longer-term goal.

6.8 Bitcoin Bridge (BTC $\rightleftharpoons$ qBTC)

qBTC is wrapped Bitcoin on RougeChain with 8 decimals (one unit is one satoshi).

- **Deposits.** Each user can request a unique Bitcoin deposit address and pay it from any wallet. Alternatively, a deposit can carry the recipient's RougeChain address in an `OP_RETURN` output. The node verifies each deposit against two independent Esplora providers and a minimum confirmation depth (default 2) before minting qBTC, and each output can be credited only once. The node watches deposit addresses but holds no Bitcoin private key.
- **Withdrawals.** The user burns qBTC. An operator-run relayer holding the custody key broadcasts the Bitcoin payout, capped per withdrawal, and the node re-verifies the payout on the Bitcoin chain before marking the withdrawal fulfilled.

Status. The Bitcoin bridge went live on mainnet on 15 September 2026. The Bitcoin relayer has been stopped since the F49 upgrade (Section 3.1.3), so the Bitcoin bridge is **paused**: qBTC withdrawals submitted now would wait unpaid until it is reopened.

7. NFT Standard

RougeChain implements a native NFT standard at the protocol level with collection-based organization and on-chain royalty enforcement.

7.1 Collections

An NFT collection defines the metadata and rules for a group of tokens:

FIELD	DESCRIPTION
collection_id	Unique identifier: <code>col:{creator_prefix}:{SYMBOL}</code>
symbol	Short identifier (e.g., "ART")
name	Human-readable name
creator	ML-DSA-65 public key of the creator
max_supply	Optional cap on total tokens
royalty_bps	Royalty percentage in basis points (0-10,000)
royalty_recipient	Address receiving royalty payments
frozen	Whether the collection is permanently frozen

7.2 Tokens

Each token within a collection has:

FIELD	DESCRIPTION
token_id	Sequential ID within the collection
owner	Current owner's public key
name	Token name
metadata_uri	URI pointing to off-chain metadata
attributes	On-chain JSON attributes
locked	Whether the token is transfer-locked

7.3 Operations

OPERATION	DESCRIPTION	FEE
Create Collection	Define a new NFT collection	50 XRGE

OPERATION	DESCRIPTION	FEE
Mint	Create a new token in a collection	5 XRGE
Batch Mint	Mint multiple tokens in one transaction	5 XRGE each
Transfer	Transfer ownership to another address	1 XRGE
Burn	Permanently destroy a token	0.1 XRGE
Lock / Unlock	Prevent or allow transfers	0.1 XRGE
Freeze Collection	Permanently prevent all operations	0.1 XRGE

7.4 Royalties

Royalties are enforced at the protocol level. When a transfer includes a `salePrice`, the royalty percentage (defined in basis points during collection creation) is automatically calculated and credited to the `royalty_recipient`. This cannot be circumvented because transfers are processed by the node, not by external contracts.

8. Encrypted Messenger and Mail

RougeChain includes end-to-end encrypted messenger and mail systems that leverage the same post-quantum primitives used by the blockchain, with comprehensive security hardening across authentication, encryption, and storage layers.

8.1 Encryption Protocol

Each messenger wallet generates two key pairs:

- **ML-DSA-65** for message signing, authentication, and request signing.
- **ML-KEM-768** for key encapsulation and encryption.

Messenger Encryption. When a user sends a message, the following process occurs:

1. **Key Encapsulation.** The sender encapsulates a shared secret using the recipient's ML-KEM-768 public key, producing a ciphertext and a 32-byte shared secret.
2. **Key Derivation.** The shared secret is passed through HKDF-SHA-256 to derive an AES-256-GCM key.
3. **Encryption.** The plaintext message is encrypted with AES-256-GCM using a random 12-byte IV.
4. **Dual Encryption.** The same process is repeated with the sender's own ML-KEM-768 public key, producing a second encrypted copy. This allows the sender to decrypt their own messages when re-fetched from the server.

5. **Signing.** The entire encrypted package is signed with the sender's ML-DSA-65 private key.

Mail Encryption (CEK Pattern). Mail uses a Content Encryption Key (CEK) pattern for multi-recipient support:

1. **CEK Generation.** A random 256-bit AES key (the CEK) is generated.
2. **Content Encryption.** The mail content (subject, body, and attachment) is encrypted once using the CEK with AES-256-GCM.
3. **Key Wrapping.** For each recipient (and the sender), the CEK is wrapped using ML-KEM-768: a shared secret is encapsulated, an AES-GCM wrapping key is derived via HKDF, and the CEK is encrypted with this wrapping key.
4. **Unified Signature.** A single ML-DSA-65 signature is computed over the concatenation of all encrypted parts (subject + body + attachment), ensuring integrity of the complete message.

This design encrypts the message body only once regardless of recipient count, making multi-recipient mail efficient while preserving per-recipient key isolation.

8.2 Authenticated API Requests

All mail, messenger, and name registry operations require ML-DSA-65 signed requests. Each API call includes:

- **Payload** — The operation parameters plus a `from` field (sender's public key), a `timestamp` (millisecond precision, valid within a 5-minute window), and a unique `nonce` (16 bytes of random hex).
- **Signature** — ML-DSA-65 signature over the canonical (sorted-key) JSON serialization of the payload.
- **Public Key** — The signer's ML-DSA-65 public key for verification.

The server verifies the signature, checks the timestamp window, confirms the `from` field matches the signing key, and rejects duplicate nonces to prevent replay attacks. Unsigned legacy endpoints return HTTP 410 (Gone) in production.

8.3 Anti-Replay Protection

Beyond timestamp validation, each signed request includes a cryptographically random nonce. The server maintains an in-memory nonce store with automatic expiry. Duplicate nonces within the validity window are rejected, preventing captured requests from being replayed even within the 5-minute timestamp tolerance.

8.4 Message Verification and TOFU

Recipients verify the ML-DSA-65 signature before decryption, ensuring message authenticity and integrity. Invalid signatures are flagged in the UI.

Trust-on-First-Use (TOFU). The messenger implements a TOFU model for key verification:

1. When a user first communicates with a contact, the contact's public key fingerprint (SHA-256 hash) is recorded in local storage.
2. On subsequent interactions, the current key fingerprint is compared against the stored value.
3. If the key has changed, a visible "Key Changed" warning is displayed in the chat header, alerting the user to potential key compromise or re-registration.
4. The shortened fingerprint is displayed in the chat header for manual out-of-band verification.

8.5 Features

- **Self-destructing messages** with configurable timers.
- **Spoiler tags** for content that should be hidden until explicitly revealed.
- **Media support** for images and video (up to 10 MB), encrypted identically to text.
- **1-on-1 and group conversations.** Direct-message conversations have deterministic IDs (`dm_` followed by a SHA-256 over the two participants' canonical wallet IDs), so both sides always open the same thread and creating a conversation twice returns the existing one.
- **Group management.** Any participant can rename a group (up to 100 characters) or add members, up to 50 participants. New members can read messages sent after they join; earlier messages were never encrypted to them.
- **Recoverable delete.** Deleting a conversation or message hides it for the deleting participant only and moves it to a trash folder. It can be restored for 30 days or purged immediately; after 30 days the node removes it.
- **Avatars.** Messenger profiles can carry an avatar image (up to 256 KB).
- **Mail folders** — Inbox, Sent, Trash, Starred, Drafts.
- **Mail threading** — Reply chains via `replyToId`.
- **Name registry** — Human-readable `@rouge.quant` / `@qwalla.mail` addresses with atomic registration.
- **Attachments** — Encrypted file attachments up to 3 MB.

8.6 Privacy by Design

Messenger and mail data is stored off-chain in a sled embedded database. The nodes store encrypted message blobs but cannot decrypt them. Only the sender and recipient, possessing the correct ML-KEM-768 private keys, can read message contents.

8.7 Client-Side Key Protection

Private keys are stored in both `sessionStorage` and `localStorage` when no password vault is configured, ensuring PWA and mobile browser sessions survive app restarts. Once the user sets a vault passphrase, plaintext keys are removed from persistent storage and only the encrypted

blob (AES-256-GCM with a PBKDF2-derived key, 600,000 iterations) remains in `localStorage`. On unlock, keys are restored to `sessionStorage` for the active session. When the vault is locked or auto-locked, all plaintext key material is cleared from both `sessionStorage` and `localStorage`.

8.8 Real-Time Delivery and Push Notifications

Real-Time Events. Clients can hold a WebSocket connection to the node and subscribe to their own inbox by sending an ML-DSA-65 signed `messenger_ws_subscribe` request. The node then pushes a `new_message` event -- conversation ID, message ID, timestamp, sender and participant IDs, but no message content -- to that caller's authenticated sockets only; an unauthenticated subscription cannot join another user's inbox topic. The client fetches and decrypts the message as usual.

Push Notifications. Mobile clients register an Expo push token with an ML-DSA-65 signed request. The node sends notifications for incoming transfers, messages, mail and group changes through Expo's push service. Because the node cannot read encrypted content, message and mail notifications carry a generic text (for example, "You received a new encrypted message"). Node operators can disable push delivery.

9. Social Layer

RougeChain includes a node-hosted social layer that provides social primitives — posts, likes, reposts, follows, comments, and play tracking — without requiring a separate protocol, token, or infrastructure deployment.

The social layer is **not on-chain**. Posts and engagement are signed with ML-DSA-65 and stored by the node in its `social-db` database; they are not included in blocks and are not covered by the state root. Only tips (Section 9.6) are on-chain transactions.

9.1 Design Rationale

Centralized social platforms control users' identity, content, and social graphs. Existing decentralized social protocols (Farcaster, Lens) address this but introduce separate infrastructure (hubs, L2 deployments) and distinct token economies. RougeChain embeds social features directly into the L1 node, following the same vertically integrated design philosophy applied to the DEX, messenger, and mail systems.

All social writes are authenticated via ML-DSA-65 signed requests using the same v2 signed API format as all other write operations on the chain. This means every post, like, follow, and repost is cryptographically attributable to the signer's public key and protected against replay attacks.

9.2 Data Model

Social data is stored in a dedicated sled embedded database (`social-db`) with the following trees:

TREE	KEY FORMAT	VALUE	PURPOSE
<code>social_posts</code>	<code>post_id (UUID)</code>	JSON <code>SocialPost</code>	Post content and metadata
<code>social_user_posts_idx</code>	<code>pubkey:reverse_timestamp</code>	<code>post_id</code>	User post index (newest first)
<code>social_post_counts</code>	<code>pubkey</code>	<code>u64 BE</code>	Per-user post count
<code>social_global_timeline</code>	<code>reverse_timestamp:post_id</code>	<code>post_id</code>	Global timeline (newest first)
<code>social_play_counts</code>	<code>track_id</code>	<code>u64 BE</code>	Per-track play count
<code>social_likes</code>	<code>target_id:pubkey</code>	<code>"1"</code>	Like membership
<code>social_like_counts</code>	<code>target_id</code>	<code>u64 BE</code>	Per-target like count
<code>social_user_likes</code>	<code>pubkey:target_id</code>	<code>"1"</code>	Reverse index for user's likes
<code>social_comments</code>	<code>comment_id (UUID)</code>	JSON <code>SocialComment</code>	Comment content
<code>social_track_comments_idx</code>	<code>track_id:timestamp</code>	<code>comment_id</code>	Track comment index
<code>social_comment_counts</code>	<code>track_id</code>	<code>u64 BE</code>	Per-track comment count
<code>social_followers</code>	<code>artist:follower</code>	<code>"1"</code>	Follow membership
<code>social_follower_counts</code>	<code>pubkey</code>	<code>u64 BE</code>	Per-user follower count
<code>social_following</code>	<code>follower:artist</code>	<code>"1"</code>	Reverse follow index
<code>social_following_counts</code>	<code>pubkey</code>	<code>u64 BE</code>	Per-user following count
<code>social_reposts</code>	<code>post_id:pubkey</code>	<code>"1"</code>	Repost membership
<code>social_repost_counts</code>	<code>post_id</code>	<code>u64 BE</code>	Per-post repost count
<code>social_reply_counts</code>	<code>post_id</code>	<code>u64 BE</code>	Per-post reply count
<code>social_reply_idx</code>	<code>parent_id:timestamp</code>	<code>child_post_id</code>	Reply threading index

Reverse timestamps (`u64::MAX - millis`) are used for index keys so that lexicographic sled iteration naturally yields newest-first ordering without explicit sorting.

9.3 Posts and Threading

Posts are standalone text entries (max 4000 characters) identified by UUID. A post optionally references a `reply_to_id`, forming a threaded reply tree. Replies increment the parent's reply count and are indexed for efficient retrieval.

```
SocialPost {  
  id: String,           // UUID  
  author_pubkey: String, // ML-DSA-65 public key  
  body: String,         // max 4000 chars  
  reply_to_id: Option<String>, // parent post for threading  
  created_at: String,   // RFC 3339 timestamp  
}
```

9.4 Timeline Feeds

Two timeline modes are supported:

- **Global timeline** — All posts ordered by creation time (newest first). Implemented as a sequential sled scan over the `social_global_timeline` tree.
- **Following feed** — Posts from users the viewer follows, merged and sorted newest-first. Implemented by scanning each followed user's post index and merging results client-side.

Both support pagination via `limit` and `offset` parameters.

9.5 Engagement Primitives

All engagement actions are toggle operations — calling the same endpoint twice reverses the action:

ACTION	ENDPOINT	EFFECT
Like	POST <code>/api/v2/social/like</code>	Toggles like on any post or track
Repost	POST <code>/api/v2/social/repost</code>	Toggles repost on any post
Follow	POST <code>/api/v2/social/follow</code>	Toggles follow on any user

Like counts, repost counts, and follower counts are maintained as atomic counters in dedicated sled trees.

9.6 Tips

Tips are not stored in the social layer — they settle directly on L1 as standard XRGV transfer transactions via `rc.transfer()`. This means tips have the same settlement, fee model, and quantum-resistant signing as any other on-chain transfer.

9.7 Security Properties

PROPERTY	MECHANISM
Authenticated writes	Every social write requires an ML-DSA-65 signed request
Anti-replay	Timestamp + nonce protection (same as all v2 endpoints)
Provable authorship	Posts are cryptographically linked to the signer's public key
No impersonation	Cannot post, like, or follow as another user without their private key
Quantum-resistant	All signatures use ML-DSA-65 (FIPS 204)
Deletion authorization	Only the original author can delete their posts or comments

10. Developer Ecosystem

10.1 TypeScript SDK

The `@rougechain/sdk` package (version 1.9 on npm) provides a TypeScript SDK for building on RougeChain.

Installation:

```
npm install @rougechain/sdk
```

Capabilities:

- Wallet generation and management (ML-DSA-65 key pairs)
- Transaction construction and client-side signing
- Token, NFT, DEX, bridge, and staking operations
- Social layer (posts, timeline, reposts, likes, follows, comments) via `rc.social`
- Encrypted mail and messenger via `rc.mail` and `rc.messenger`, including real-time subscriptions, group management (`updateConversation`, `addParticipants`), and trash/restore/purge
- Balance and state queries, including a one-call validator status check (`getValidatorStatus`)
- Player-signed contract deployment, calls, read-only queries and events via `rc.contracts` (Section 10.5)

Environment support: Browser, Node.js, and React Native. The SDK uses `@noble/post-quantum` for all cryptographic operations, with no native dependencies.

10.2 Browser Extension

The RougeChain browser extension (Manifest V3, version 1.4) provides:

- **Wallet management** with password-encrypted storage (PBKDF2 + AES-256-GCM).
- **Five integrated tabs:** Wallet, Tokens, NFTs, Chat (messenger), and Settings.
- **Encrypted backups:** wallet export produces a password-encrypted `.pqcbbackup` file (PBKDF2 with 600,000 iterations + AES-256-GCM) that the web wallet and Qwalla can import.
- **Real-time messenger** notifications over the node's authenticated WebSocket (Section 8.8).
- **Contract signing:** dApps can ask the extension to sign contract deployments and calls, which it signs locally after user approval (Section 10.5).
- **Smart API caching** with TTL-based deduplication to minimize network overhead.

dApp Provider. The extension injects `window.rougechain` into web pages, enabling dApps to interact with the user's wallet:

```
const { publicKey } = await window.rougechain.connect();
const { balance } = await window.rougechain.getBalance();
const { txId } = await window.rougechain.sendTransaction(payload);
```

Connected sites are tracked and require explicit user approval via a popup window. All transactions submitted through the provider are signed client-side using `m1_dsa65.sign()` before being sent to the node.

Provider Authenticity. The injected provider object carries a `Symbol.for("rougechain:authentic")` token that dApps can check to verify they are communicating with the genuine extension, not a malicious imitation. The provider is defined with `Object.defineProperty` (non-writable, non-configurable) and unconditionally overwrites any pre-existing definitions to prevent injection attacks.

10.3 REST API

Every node exposes a comprehensive HTTP API supporting all chain operations:

CATEGORY	ENDPOINTS
Chain	<code>/api/stats</code> , <code>/api/blocks</code> , <code>/api/blocks/:height</code>
Wallet	<code>/api/balance/:pubkey</code> , <code>/api/faucet</code>
Transactions	<code>/api/tx/submit</code> , <code>/api/v2/tx/submit</code> , <code>/api/tx/broadcast</code>
Tokens	<code>/api/tokens</code> , <code>/api/token/create</code> , <code>/api/token/metadata</code>
DEX	<code>/api/pools</code> , <code>/api/swap</code> , <code>/api/pool/create</code> , <code>/api/pool/:id/earnings/:owner</code>

CATEGORY	ENDPOINTS
NFTs	<code>/api/nft/collections</code> , <code>/api/nft/owner/:pubkey</code> , <code>/api/v2/nft/*</code>
Bridge	<code>/api/bridge/claim</code> , <code>/api/bridge/withdraw</code>
Contracts	<code>/api/v2/contract/publish</code> , <code>/api/v2/contract/execute</code> , <code>/api/contract/:addr/query</code> , <code>/api/contract/:addr/events</code>
Rollup	<code>/api/v2/rollup/status</code> , <code>/api/v2/rollup/submit</code> , <code>/api/v2/rollup/batch/:id</code>
Messenger	<code>/api/v2/messenger/wallets/register</code> , <code>/api/v2/messenger/conversations</code> , <code>/api/v2/messenger/messages</code>
Names	<code>/api/v2/names/register</code> , <code>/api/v2/names/release</code> , <code>/api/names/resolve/:name</code> , <code>/api/names/reverse/:walletId</code>
Mail	<code>/api/v2/mail/send</code> , <code>/api/v2/mail/folder</code> , <code>/api/v2/mail/message</code> , <code>/api/v2/mail/read</code> , <code>/api/v2/mail/move</code> , <code>/api/v2/mail/delete</code>
Social	<code>/api/social/timeline</code> , <code>/api/social/post/:id</code> , <code>/api/social/user/:pubkey/posts</code> , <code>/api/v2/social/post</code> , <code>/api/v2/social/like</code> , <code>/api/v2/social/repost</code> , <code>/api/v2/social/follow</code> , <code>/api/v2/social/feed</code>
P2P	<code>/api/peers</code> , <code>/api/peers/register</code> , <code>/api/blocks/import</code>

Mainnet availability. On the public mainnet endpoint (`api.rougechain.io`), the unauthenticated write routes `POST /api/tx/broadcast`, `POST /api/blocks/import`, `POST /api/peers/register`, `/api/v2/contract/deploy` and `/api/v2/contract/call` are not publicly open. Transactions are submitted through the signed `/api/v2/*` routes.

10.4 gRPC

A gRPC interface is available for high-performance node-to-node communication and advanced integrations, supporting the same operations as the REST API with protocol buffer serialization.

10.5 WASM Smart Contracts

RougeChain includes a fuel-metered WebAssembly (WASM) smart contract runtime built on the `wasmi` pure-Rust interpreter (used by Parity/Substrate).

Execution Model:

PARAMETER	VALUE
Runtime	<code>wasmi</code> (pure Rust, no JIT)
Fuel limit (per call)	10,000,000 instructions
Fuel limit (per block)	100,000,000 instructions
Max WASM module size	1 MB

PARAMETER	VALUE
Max call depth	8 (cross-contract)

Contract Lifecycle:

1. **Deploy** — Compile and validate WASM bytecode, compute contract address as `SHA-256(deployer || nonce)[0..20]`, store metadata and bytecode in `contracts-db`.
2. **Execute** — Call a contract method with fuel metering. State changes are committed atomically on success; rolled back entirely on failure.
3. **Query** — Read-only contract calls that do not commit state changes.

Player-Signed Transactions (GAME_READY, height 150). A contract deployment or call is valid only if the deployer or caller signed it; the signer is the caller the contract sees (`host_get_caller`) and pays the fee. Node-signed contract transactions -- whose "caller" was an unsigned field -- are invalid from height 150. A deployment costs a flat **10 XRGE**, and a call costs its signed gas limit × **0.000001 XRGE**, paid up front. The contract address is the first 20 bytes of `SHA-256("rougechain/contract/v2" || from || 0 || nonce || 0 || SHA-256(wasm))`, all of which is inside the deployer's signature, so nobody else can claim or pre-empt it. (The address rule in the lifecycle above applies to earlier, node-signed deployments.)

Host API. Contracts interact with chain state through imported host functions:

- **Core:** `host_log`, `host_get_caller`, `host_get_self_addr`, `host_get_block_height`, `host_get_block_time`, `host_get_balance`, `host_transfer` (XRGE), `host_storage_read`, `host_storage_write`, `host_storage_delete`, `host_emit_event`, `host_set_return`, `host_call_contract`, `host_get_call_result`, and the ML-DSA-65 helpers `host_pqc_verify`, `host_pqc_pubkey_to_address` and `host_pqc_hash_pubkey`.
- **Call arguments (GAME_READY, height 150):** `host_get_args_len` and `host_read_args` return the call's JSON arguments.
- **Assets and randomness (GAME_READY 2, height 160):** `host_token_balance` and `host_token_transfer` (custom tokens the contract holds), `host_nft_owner` and `host_nft_transfer` (NFTs the contract owns), `host_nft_create_collection` (a collection `col:<contract address prefix>:<SYMBOL>` owned by the contract) and `host_nft_mint` (the contract mints from its own collections to players), and `host_random`. Before height 160 these functions are not linked, so a module importing them fails to run.

From height 160, addresses a contract passes to these functions are canonicalised -- paying `host_get_caller()`, a public key, credits the player's `rouge1` ledger entry -- and NFT owner checks compare canonical addresses. The VM reads token balances and NFTs from a snapshot, records each accepted operation, and the node applies the operations in order only when the call succeeds; invalid operations make the block invalid.

Randomness. `host_random` returns 32 bytes derived from `SHA-256("rougechain/rand/v1" || parent block hash || tx hash)` and a per-call counter. The sender knows the parent hash before sending and controls the transaction's bytes, so a one-step roll can be ground: the sender signs many

variants offline, computes each result with the public VM, and sends only a winner. `host_random` must therefore never decide anything of value on its own. From height 170, `host_block_hash(height)` returns the hash of a finished block up to 256 back; games commit in one call (recording height H) and settle in a later call from the hash of block $H+1$ mixed with the player and H . That hash covers the producer's ML-DSA-65 signature and the validators' finality signatures, which the player cannot predict or choose. The single block producer could still bias an outcome by withholding a block; validator-generated (VRF) randomness is planned (Section 12). The `loot_roll` example uses this commit-then-settle pattern.

Cross-Contract Calls. Contracts can invoke other contracts up to 8 levels deep. Before height 160, when a call made cross-contract calls, none of its XRGE moves were applied (only single-hop calls moved XRGE); earlier versions of this paper wrongly said sub-call balance changes were propagated. From height 160, cross-contract calls are multi-hop: each sub-call sees every move made before it (by its caller and by earlier sub-calls), the XRGE, token and NFT moves of successful sub-calls are merged into the top-level result and applied with it, and a failed sub-call's moves are dropped. Each sub-call draws its own randomness.

Receipts, Events and Queries. A contract-call receipt reports `Failed` with the error when the call reverted; a failed call moves no assets. Contract events are delivered live over the node's WebSocket on the topic `contract:<addr>` after a block is accepted, and can be paged with `GET /api/contract/:addr/events`. `POST /api/contract/:addr/query` runs a read-only dry run against the live ledger without a signature or a transaction.

Development Workflow: Contracts are written in Rust, compiled to WASM via `wasm32-unknown-unknown`, and deployed with a signed transaction. The SDK's `rc.contracts` (version 1.9) and the browser extension (version 1.4) sign deployments and calls. The repository's `loot_roll` example contract pays prizes from its own treasury using token transfers, NFT minting and `host_random`.

Mainnet Status. Contracts can hold and move XRGE since height 18, and custom tokens and NFTs since height 160. Signed deployments and calls are submitted through `/api/v2/contract/publish` and `/api/v2/contract/execute`. The older unsigned `/api/v2/contract/deploy` and `/api/v2/contract/call` routes remain closed on the public mainnet endpoint. Storage is loaded eagerly and there is no storage fee yet.

10.6 MCP Server

`@rougechain/mcp-server` (version 1.2.0) exposes RougeChain to AI agents over the Model Context Protocol. By default it is read-only (32 tools), including free contract queries (`POST /api/contract/:addr/query`), contract events and transaction receipts. When a wallet is supplied through environment variables, it also registers 31 write tools -- transfers, staking, tokens, DEX, NFTs, names, social, bridge withdrawals, and player-signed contract deployment and calls (`/api/v2/contract/publish` and `/execute`, Section 10.5) -- and signs each one locally with ML-DSA-65; the key never leaves the process. Governance voting and messenger/mail sending are not exposed.

10.7 Command-Line Interface

The `rougechain` CLI signs transactions locally with ML-DSA-65 and supports key generation, transfers, staking and messenger operations. For validator operators it can sign directly with a node's `node-keys.json` (`--node-keys`) and provides `rougechain validator-status` , a one-shot check that a key is funded, staked, in the active set and producing blocks.

10.8 Applications

- **rougechain.io** -- web wallet, explorer, DEX, bridges, messenger and mail, a swap for buying XRGE on Base, and a network status page. The interface is available in English, Spanish, Chinese and Japanese.
 - **Qwalla** -- a wallet suite built from one Expo/React Native codebase: the **iOS app** (App Store), the **web app** at qwalla.io, **desktop apps** for macOS and Windows, and **Qwalla Browser**, a standalone desktop browser for on-chain apps. All share the same tokens, messenger, mail and dApp access; the iOS app adds push notifications.
 - **rougee.app (qRougee)** -- a decentralized music dApp that connects to the browser extension through the `window.rougechain` provider.
 - **rougecoin.io** -- information site for XRGE, linked from rougechain.io.
-

11. Security Considerations

11.1 Cryptographic Security

RougeChain targets NIST Level 3 security for signatures and encryption (~192-bit classical, ~128-bit quantum), while STARK proofs provide ~100-bit soundness -- all of which remain well beyond practical attack feasibility. The lattice-based problems underlying ML-DSA-65 and ML-KEM-768 have been studied extensively and are considered resistant to all known quantum and classical attacks. Blake3, used for STARK commitments, is not itself NIST-standardized but provides equivalent collision resistance; the overall proof system's security reduces to hash collision resistance, which is unaffected by Shor's algorithm.

11.2 Client-Side Key Management

Private keys are generated and stored exclusively on the user's device. The browser extension encrypts keys at rest using PBKDF2 (600,000 iterations) with AES-256-GCM. Transaction signing occurs locally; only the signed transaction is transmitted to the network.

Legacy API endpoints that previously accepted raw private keys for server-side signing are disabled in production and return HTTP 410 (Gone). They are accessible only in local development mode via the `--dev` flag.

11.3 Validator Accountability

Validators stake at least 10,000 XRGE, and the slash count and jail status of each validator are recorded in validator state. Slashing confiscates 10% of stake and jails the validator for 20 blocks, during which it is not eligible to propose. Automatic slashing for missed blocks applied until height 100 and is frozen under Release 1 proposer selection (Section 3.1). Slashing for provable misbehavior, such as equivocation, is planned as Release 3 and is not active; until then, the proposal journal prevents an honest producer from equivocating, but a dishonest one is not penalized automatically.

11.4 Network Resilience

- **Mempool limits** (2,000 transactions) prevent memory exhaustion.
- **Three-tier rate limiting** with cryptographic validator authentication protects all endpoints.
- **Block pagination** caps API responses to prevent unbounded data dumps.
- **Exponential backoff** on peer sync failures prevents cascade overloads.
- **Adaptive polling** reduces unnecessary network traffic during quiet periods.
- **Transaction-identity uniqueness** (from height 90) prevents a signed transaction from being included twice, at mempool admission, block production and block import.

11.5 Block Import Verification

When importing blocks from peers, the node performs the following verification:

1. **Height continuity** -- The block must extend the current tip by exactly one.
2. **Hash chain** -- The block's `prev_hash` must match the current tip hash.
3. **Proposer signature** -- The `proposer_sig` is verified against `proposer_pub_key` using `pgc_verify`, and the block hash is recomputed to ensure integrity.
4. **Proposer eligibility** -- Before height 100, the proposer had to be an actively staked validator. From height 100, the proposer must be the designated proposer for that height (Section 3.1); any other block is rejected before state is touched.
5. **Transaction signatures** -- All transactions are verified in parallel (using Rayon) across three signature formats (V2 signed payload, V1 new format, V1 legacy format). If any transaction fails all three verification methods, the entire block is rejected. From height 90, V2 transactions must also match their signed payload.
6. **Transaction uniqueness** -- From height 90, a block containing a transaction that is already included in an earlier block is rejected.
7. **State root** -- From height 18, the node executes the block and rejects it if the resulting ledger state root does not match the header. From height 160 the root is state root v2, which also covers NFT collections and ownership, contract code and contract storage.
8. **Pinned history** -- Blocks 18-48 must match the hashes pinned by the F49 upgrade.
9. **Parent commit certificate** -- From height 151, the block must carry `parent_commit`, a FINALITY_V2 proof for exactly its parent; the node verifies every precommit signature and recomputes the quorum from the parent's validator set, and rejects the block without

storing anything if the certificate is missing, mismatched or insufficient. A verified certificate is persisted, finalizing the parent.

10. **Contract signatures** -- From height 150, `contract_deploy` and `contract_call` must be signed by the deployer or caller; node-signed contract transactions are rejected.

11.6 Bridge Security

- **Two-layer deposit verification:** EVM receipt validation (tx status, recipient, sender, confirmations) combined with SHA-256 commitment nullifiers to prevent double-claiming.
- **Deposit event verification** for contract deposits via transaction log parsing.
- **ECDSA signature verification** (ecrecover) authenticates that the claimant is the original depositor on manual claims.
- **Chain ID validation** ensures claims target the correct network (Base Mainnet 8453 or Sepolia 84532).
- **Confirmation requirements** prevent claims against unconfirmed transactions (default: 6 confirmations on Base, 2 on Bitcoin).
- **Replay protection** via persistent commitment nullifier tracking ensures each deposit can only be claimed once.
- **Signed withdrawal intent:** the node executes only the ML-DSA-65 signed amount, fee, asset and destination; the fee must be exactly 0.1 XRGE.
- **Burn-gated payouts:** a payout record exists only if the burn succeeded in an accepted block; the node verifies the payout on the destination chain before marking it fulfilled; automatic refunds are disabled.
- **On-chain limits:** XRGE releases from BridgeVaultV2 are bounded by per-transaction and 24-hour caps set by a Safe multisig owner.
- **Hardened relayer** with payout-state checks, explicit asset routing, nonce management, exponential retry, gas estimation, preflight checks and graceful shutdown.
- **Relayer authentication** via shared secret restricts withdrawal fulfillment to authorized operators.

These measures do not remove operator trust. `bridge_mint` is authorized by the operator's key, the relayers are operator-run, and RougeBridge (ETH and USDC) is owned by a single hot key; see Section 6.7.

11.7 Messenger and Mail Security

Messages and mail are encrypted end-to-end using post-quantum key encapsulation. The server stores only ciphertext and cannot decrypt message contents. The following security measures protect the communication layer:

- **Signed API requests.** All 16 mail/messenger/name registry endpoints require ML-DSA-65 signed requests with timestamp validation and nonce-based anti-replay protection.
- **Authorization checks.** The server verifies that the signing key belongs to a registered wallet and that the caller is authorized for the requested operation (e.g., sender owns the mailbox, caller is a conversation participant).

- **Unified mail signatures.** Mail messages are signed over the concatenation of all encrypted parts (subject, body, and attachment), preventing partial content substitution.
- **Multi-recipient CEK pattern.** Mail content is encrypted once with a random CEK, which is then KEM-wrapped individually for each recipient, ensuring efficient multi-recipient delivery without re-encrypting the content.
- **TOFU key verification.** Public key fingerprints (SHA-256) are recorded on first use and compared on subsequent interactions, with visible warnings when a contact's key changes.
- **Atomic name registration.** The name registry uses sled's compare-and-swap operation to prevent race conditions during name claims.
- **Input validation.** Server-side length limits are enforced on all fields (display names: 50 chars, message content: 2 MB, mail subject: 10 KB, mail body: 512 KB, attachment: 3 MB, max 50 recipients).
- **Sled-backed storage.** Messenger data is stored in a sled embedded database with per-record atomic operations, replacing the previous JSON file storage which was susceptible to race conditions under concurrent access.

11.8 dApp Provider Security

The browser extension's injected provider (`window.rougechain`) defends against page-level tampering:

- **Unconditional injection** overwrites any malicious pre-definitions.
- **Non-writable, non-configurable** property descriptor prevents post-injection modification.
- **Authenticity token** (`Symbol.for("rougechain:authentic")`) allows dApps to verify they are communicating with the genuine extension.
- **Approval popups** require explicit user consent for connect, sign, and send operations.

11.9 Zero-Knowledge Proof Security

The zk-STARK proof system provides an additional security dimension: transaction privacy without sacrificing verifiability.

- **Soundness.** A malicious prover cannot construct a valid proof for an invalid statement (e.g., a transfer that creates value from nothing). The FRI protocol ensures that forging a proof requires finding collisions in Blake3-256, which is computationally infeasible.
- **Zero-knowledge.** The verifier learns nothing beyond the validity of the statement. Initial balances and transfer amounts are never exposed. The proof transcript is statistically indistinguishable from a simulated transcript.
- **Transparency.** Unlike zk-SNARKs, STARKs require no trusted setup ceremony. There is no "toxic waste" -- the system's security does not depend on any party honestly destroying secret parameters.
- **Post-quantum safety.** The proof system's security reduces to the collision resistance of Blake3-256, which is unaffected by Shor's algorithm and only quadratically weakened by Grover's algorithm (to ~128-bit security).

11.10 Current Limitations

RougeChain is an early-stage network. As of 28 September 2026:

- **One producing validator.** Three accounts are staked, but only the operator's validator (the largest stake) produces blocks, and under Release 1 it is the designated proposer at every height; the second mainnet node follows the chain without producing blocks. If the producer is offline, the chain stops until it returns.
- **Finality without liveness.** FINALITY_V2 has been active since height 150, but the operator's validator holds enough stake to finalize each block with its own vote. Release 2b -- a fallback proposer, skip certificates, slashing tied to it, and spreading stake across independently run validators -- is not implemented (Section 3.1).
- **Contract randomness.** One-step `host_random` rolls can be ground by the sender; games must commit and settle with `host_block_hash` (height 170). The single producer can still bias a settlement by withholding a block; validator VRF randomness is planned, not built (Section 10.5).
- **Operator authority.** The operator's key authorizes bridge mints, the bridge relayers are operator-run, protocol upgrades are coordinated binary releases without on-chain governance, and most of the XRGE supply is held in operator-controlled protocol accounts.
- **Bridges are operator-trusted.** RougeBridge (ETH and USDC) is owned by a single hot key. The Bitcoin bridge is paused (Section 6.8).
- **Post-quantum bridge not deployed.** BridgeVaultV3 is an audit candidate only; its FINALITY_V2 prerequisite is now active, but it awaits an external audit.
- **No external audit.** Security reviews to date have been internal.
- **Networking.** Nodes communicate by HTTP polling; peer-write routes are closed on the public endpoint, and a peer-to-peer transport is not yet implemented.

12. Roadmap

12.1 Shipped

FEATURE	NOTES
zk-STARK balance transfer proofs	
Shielded transactions with nullifier sets	
ZK-rollup batch proving	
WASM smart contract runtime (wasmi, fuel-metered)	
WASM-compiled STARK prover for browser	

FEATURE	NOTES
On-chain governance with delegation and treasury	
Multi-signature wallets (M-of-N)	
Unbonding period and slashing machinery	Missed-block auto-slashing frozen from height 100
JSON-RPC 2.0 (eth_*/rouge_* dual namespace)	
WebSocket real-time event subscriptions	Including authenticated private messenger events
Genesis configuration and mainnet bootstrapping	rougechain-mainnet-1
Event indexer (sled-backed, multi-index)	
CLI wallet (ML-DSA-65 signed transactions)	Including validator-status
Prometheus metrics and Docker containerization	
Mail/Messenger/Names security hardening (signed requests, anti-replay, CEK, TOFU, sled)	
Social layer (posts, timeline, reposts, likes, follows, comments, tips)	Node-hosted
Native limit orders	
v2 integer ledger and state root	Height 18
F49 ledger and bridge-withdrawal hardening	Height 49
Transaction integrity (identity uniqueness + signed-payload binding)	Height 90
Proposer selection, Release 1	Height 100
FINALITY_V2 verified finality (Release 2a: parent commit certificates)	Height 150
GAME_READY: player-signed contract deployment and calls	Height 150
GAME_READY 2: contracts hold/move tokens and NFTs, mint NFTs, host_random, multi-hop cross-contract moves, state root v2	Height 160
GAME_READY 3: host_block_hash for commit-then-settle randomness	Height 170
Contract events over WebSocket, contract query endpoint, loot_roll example	
LP fee collection ("Collect fees")	Node-side fee ledger; no protocol fee
Base bridge with BridgeVaultV2 (Safe owner, capped relayer) for XERGE	ETH/USDC still on RougeBridge (hot-key owner)
BTC ⇌ qBTC bridge	Live 15 September 2026; currently paused

FEATURE	NOTES
Messenger groups, avatars, deterministic DMs, recoverable delete	
Push notifications	
TypeScript SDK 1.9 (<code>rc.contracts</code>), browser extension 1.4 (contract signing), MCP server 1.1	

12.2 In Progress

FEATURE	STATUS
Proposer selection, Release 2b (skip certificates + deterministic fallback proposer, validator-admission hardening)	Designed, not implemented; needs stake spread across more validators to matter
BridgeVaultV3 post-quantum bridge (M-of-N ML-DSA-65 authorization on Base)	Audit candidate frozen and rehearsed on a test network; external audit next; not deployed
External security audit	Not yet started
Decentralization: independent validators and a peer-to-peer transport	One producing validator today
RougeBridge ownership moved to a multisig	Pending
Bitcoin bridge reopening	Paused since the F49 upgrade

12.3 Planned

FEATURE
Proposer selection, Release 3 (slashing on equivocation evidence)
Validator VRF randomness for contracts (after Release 2b)
Contract storage fees and lazy storage loading; incremental (Merkle) state root
On-chain protocol upgrade governance
Concentrated liquidity (range-based positions)
Yield farming (LP staking rewards)
On-chain NFT marketplace
Fully trustless STARK bridge (Base light client)
SLH-DSA (SPHINCS+) as alternative signature scheme
Hybrid classical + PQC mode
Hardware wallet support

FEATURE

Threshold signatures for multi-sig

v2.0 removed two items that v1.8 had marked complete: **BFT finality** (the legacy implementation did not verify votes) and **STARK bridge deposit verification** (Base deposits are verified through RPC receipts, not STARK proofs). Verified finality returned to the Shipped list with FINALITY_V2 at height 150; STARK bridge verification remains a planned item.

References

1. NIST. *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)*. August 2024.
 2. NIST. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*. August 2024.
 3. Shor, P.W. *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer*. SIAM Journal on Computing, 1997.
 4. Grover, L.K. *A Fast Quantum Mechanical Algorithm for Database Search*. Proceedings of the 28th Annual ACM Symposium on Theory of Computing, 1996.
 5. Adams, J., et al. *Uniswap V2 Core*. Uniswap, 2020.
 6. Ben-Sasson, E., et al. *Scalable, transparent, and post-quantum secure computational integrity*. IACR Cryptology ePrint Archive, 2018.
 7. Facebook/Meta. *winterfell: A STARK prover and verifier library*. GitHub, 2024.
<https://github.com/facebook/winterfell>
 8. Ben-Sasson, E., et al. *Fast Reed-Solomon Interactive Oracle Proofs of Proximity (FRI)*. ICALP 2018.
-

RougeChain is open-source software. The complete codebase, including the Rust core daemon, TypeScript SDK, browser extension, and web frontend, is publicly available.